Lab Report

Network Socket Programming & Packet Analysis

Omar Adel Abdallah
ID: 8746

Contents

1 Introduction **2**

2 TCP Server and Client Implementation **2**
2.1 TCP Server Code 2
2.2 TCP Client Code 3
2.3 TCP Execution Screenshot 5

3 UDP Server and Client Implementation **6**
3.1 UDP Server Code 6
3.2 UDP Client Code 7
3.3 UDP Execution Screenshot 8

4 Wireshark Packet Capture Analysis **9**
4.1 TCP 3-Way Handshake 9
4.2 UDP Packet Capture 9

5 Conclusion **10**

1 Introduction

This report documents the implementation of custom client-server architectures using Python's `socket` library. The lab covers both connection-oriented (TCP) and connectionless (UDP) protocols, followed by network traffic analysis using Wireshark via the local loopback interface.

2 TCP Server and Client Implementation

In this section, a multi-threaded TCP server was created to handle concurrent client connections. A custom header protocol was used to dictate message lengths.

2.1 TCP Server Code

```python
import socket
import threading

PORT = 5050
HEADER = 64
DISCONNECT_MESSAGE = "!DISCONNECTED"
INVALID_MESSAGE = "!INVALID MESSAGE"
FORMAT = "utf-8"
SERVER = socket.gethostbyname(socket.gethostname())
ADDR = (SERVER, PORT)

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(ADDR)

def handle_request(conn, addr):
    print(f"[NEW CONNECTION] {addr} connected.")
    connected = True

    try:
        while connected:
            msg_length_str = conn.recv(HEADER).decode(FORMAT)

            if not msg_length_str:
                break

            msg_length = int(msg_length_str)
            msg = conn.recv(msg_length).decode(FORMAT)

            if msg == DISCONNECT_MESSAGE:
                connected = False
                print(f"[{addr}] Disconnected.")
            else:
                if len(msg) > 0:
                    response = str(msg[1:].strip())

                    if (msg[0] == 'A'):
                        response = "".join(filter(lambda x:x.isalpha(), sorted(response,key=lambda x:x.lower(), reverse=True)))
                    elif (msg[0] == 'C'):
```

```python
                        response = "".join(filter(lambda x:x.isalpha(), sorted(response,key=lambda x:x.lower())))
                    elif (msg[0] == 'D'):
                        response = response.upper()
                    else:
                        response = msg[0] + response
                else:
                    response = INVALID_MESSAGE

                conn.send(str(response).encode(FORMAT))

            print(f"[{addr}] {msg}")

    except ConnectionResetError:
        print(f"[ERROR] Connection with {addr} was lost.")
    except Exception as e:
        print(f"[ERROR] An unexpected error occurred with {addr}: {e}")
    finally:
        conn.close()
        print(f"[DISCONNECTED] {addr} connection closed. Active connections: {threading.active_count() - 2}")

def start():
    server.listen()
    print(f"[TCP SERVER IS LISTENING] on {SERVER}:{PORT}")

    while True:
        conn, addr = server.accept()
        thread = threading.Thread(target=handle_request, args=(conn, addr))
        thread.start()
        print(f"[ACTIVE CONNECTIONS] {threading.active_count() - 1}")

print("[STARTING] TCP server is starting.....")
start()
```

Listing 1: TCP Server Implementation

2.2 TCP Client Code

```python
import socket
import threading
import random
import time
import string

PORT = 5050
HEADER = 64
FORMAT = "utf-8"
DISCONNECT_MESSAGE = "!DISCONNECTED"
SERVER = "127.0.1.1"
ADDR = (SERVER, PORT)

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

client.connect(ADDR)

def send(msg):
```

```python
    message = msg.encode(FORMAT)
    msg_length = len(message)
    send_length = str(msg_length).encode(FORMAT)
    send_length += b' ' * (HEADER - len(send_length))
    client.send(send_length)

    client.send(message)

    if msg != DISCONNECT_MESSAGE:
        received_msg = client.recv(2048).decode(FORMAT)
        print(f"[CLIENT SENT] {msg}")
        print(f"[SERVER RESPONSE] {received_msg}\n")

def generate_random_string(length = 10):
    first_chars = ['A', 'C', 'D', 'E', 'F']
    start_char = random.choice(first_chars)

    pool_of_characters = string.ascii_letters + string.digits
    rest_of_string = ''.join(random.choice(pool_of_characters) for _ in
    range(length - 1))

    return start_char + rest_of_string

print("[CLIENT STARTING] Sending random messages...\n")

for i in range(5):
    random_msg = generate_random_string(random.randint(8, 15))
    send(random_msg)
    time.sleep(3)

print("[CLIENT DISCONNECTING]...")
send(DISCONNECT_MESSAGE)
```

Listing 2: TCP Client Implementation

2.3 TCP Execution Screenshot

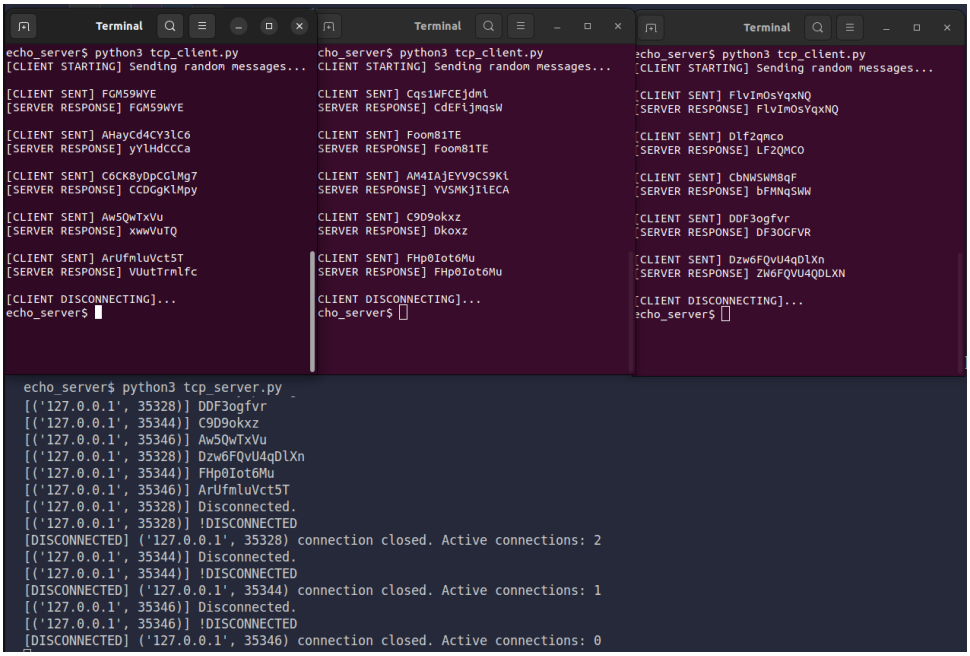

Figure 1: Terminal output demonstrating successful TCP communication.

3 UDP Server and Client Implementation

The architecture was subsequently converted to use the User Datagram Protocol (UDP). Because UDP is connectionless, the header-parsing logic was removed in favor of direct datagram transmission using `recvfrom()` and `sendto()`.

3.1 UDP Server Code

```python
import socket
import threading

PORT = 5050
HEADER = 64
BUFFER_SIZE = 1024
DISCONNECT_MESSAGE = "!DISCONNECTED"
INVALID_MESSAGE = "!INVALID MESSAGE"
FORMAT = "utf-8"
SERVER = socket.gethostbyname(socket.gethostname())
ADDR = (SERVER, PORT)

print(SERVER)
server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind(ADDR)

def handle_message(msg_bytes, addr):
    try:
        msg = msg_bytes.decode(FORMAT)
        if msg == DISCONNECT_MESSAGE:
            print(f"[{addr}] Sent disconnect notification.")
            return

        if len(msg) > 0:
            response = str(msg[1:].strip())

            if (msg[0] == 'A'):
                response = "".join(filter(lambda x:x.isalpha(), sorted(response,key=lambda x:x.lower(), reverse=True)))
            elif (msg[0] == 'C'):
                response = "".join(filter(lambda x:x.isalpha(), sorted(response,key=lambda x:x.lower())))
            elif (msg[0] == 'D'):
                response = response.upper()
            else:
                response = msg[0] + response
        else:
            response = INVALID_MESSAGE

        server.sendto(str(response).encode(FORMAT), addr)
        print(f"[{addr}] message: {msg}")

    except Exception as e:
        print(f"[ERROR] An unexpected error occurred with {addr}: {e}")

def start():
    print(f"[UDP SERVER IS LISTINING] on {SERVER}:{PORT}")

```

```
    while True:
        data, addr = server.recvfrom(BUFFER_SIZE)
        thread = threading.Thread(target=handle_message, args=(data, addr))
        thread.start()

print("[STARTING] UDP server is starting.....")
start()
```

Listing 3: UDP Server Implementation

3.2 UDP Client Code

```
import socket
import random
import string
import time
import sys

PORT = 5050
FORMAT = "utf-8"
DISCONNECT_MESSAGE = "!DISCONNECTED"
SERVER = "127.0.1.1"
ADDR = (SERVER, PORT)

client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)


def send(msg):
    try:
        message = msg.encode(FORMAT)

        client.sendto(message, ADDR)

        if msg != DISCONNECT_MESSAGE:
            client.settimeout(2.0)
            data, server_addr = client.recvfrom(2048)
            print(f"[SERVER RESPONSE] {data.decode(FORMAT)}\n")

    except socket.timeout:
        print("\n[ERROR] Server took too long to respond. Packet dropped!\n")
    except Exception as e:
        print(f"\n[ERROR] Network error: {e}")
        sys.exit()

def generate_random_string(length=10):
    first_chars = ['A', 'C', 'D', 'X', 'Z']
    start_char = random.choice(first_chars)
    pool_of_characters = string.ascii_letters + string.digits
    rest_of_string = ''.join(random.choice(pool_of_characters) for _ in range(length - 1))
    return start_char + rest_of_string


print("[CLIENT STARTING] Sending UDP packets...\n")

for i in range(5):
```

```python
    random_msg = generate_random_string(random.randint(8, 15))
    print(f"[CLIENT SENDING] {random_msg}")
    send(random_msg)
    time.sleep(3)

print("[CLIENT DISCONNECTING]...")
send(DISCONNECT_MESSAGE)
```

Listing 4: UDP Client Implementation

3.3 UDP Execution Screenshot

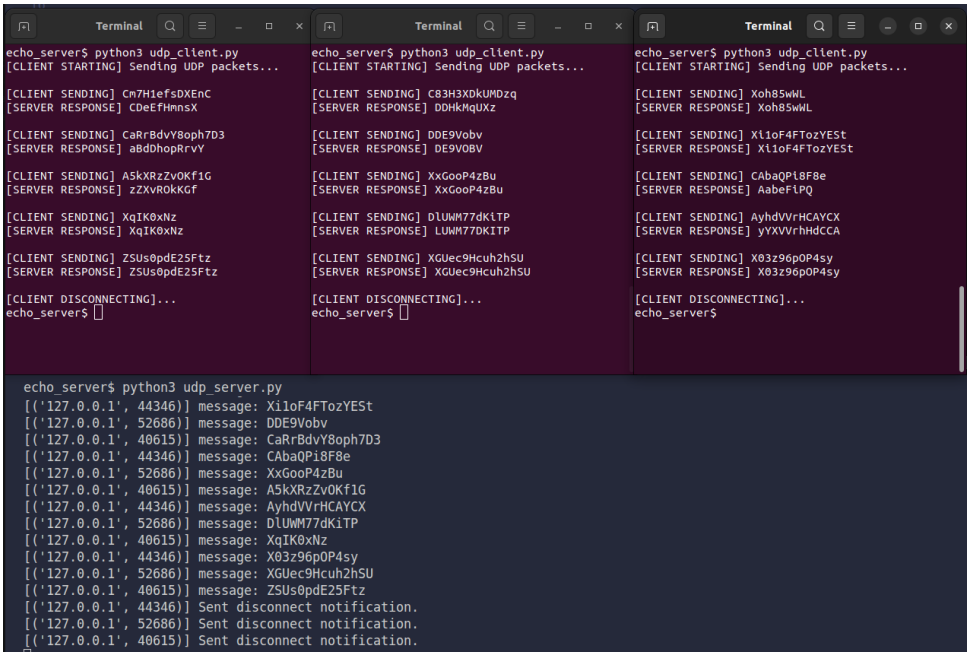

Figure 2: Terminal output demonstrating successful UDP datagram exchange.

4 Wireshark Packet Capture Analysis

To verify the underlying network behavior, traffic was captured on the `lo` (loopback) interface using Wireshark. Display filters for both TCP and UDP ports were applied to isolate the relevant application traffic.

4.1 TCP 3-Way Handshake

The following capture demonstrates the standard TCP connection establishment (SYN, SYN-ACK, ACK) generated by the `client.connect()` method.

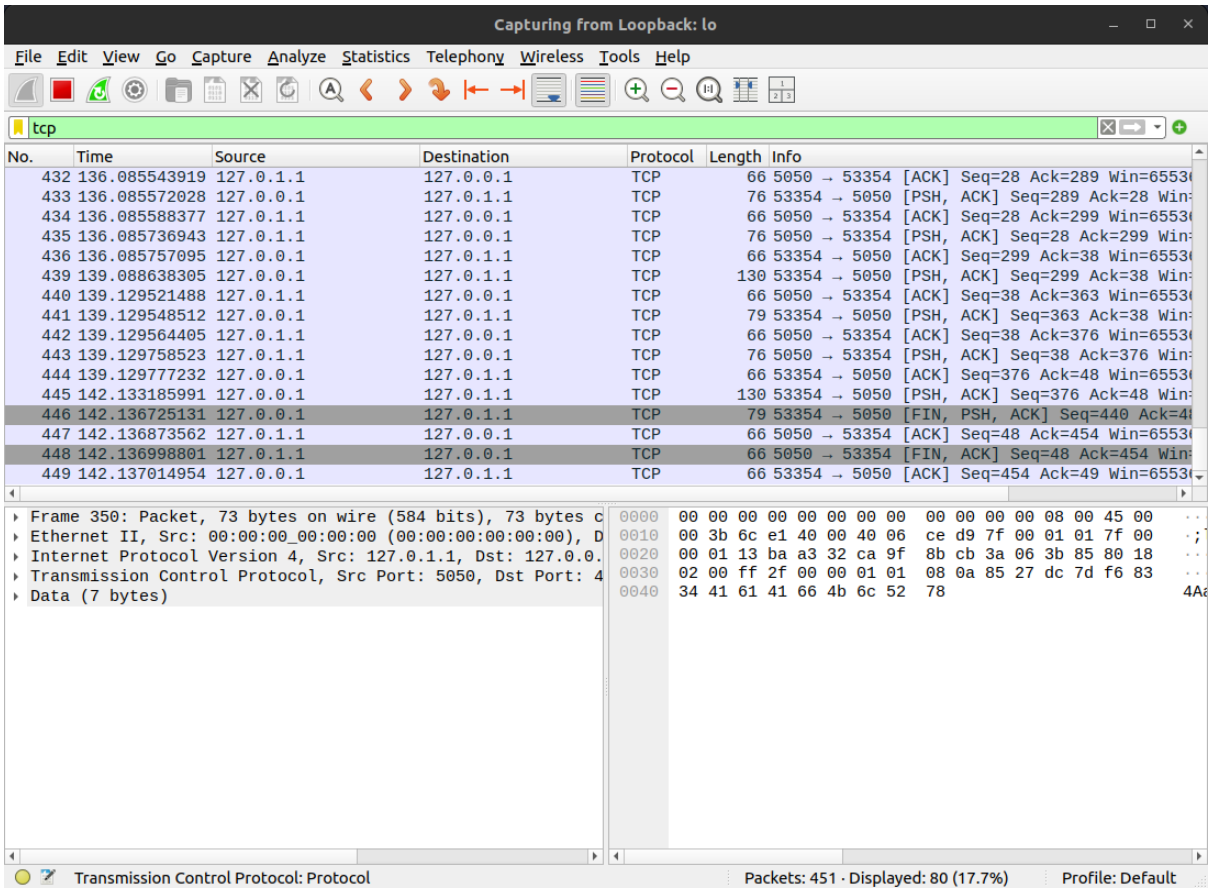

Figure 3: Wireshark capture of the TCP 3-way handshake.

4.2 UDP Packet Capture

The following capture demonstrates connectionless UDP datagram transmission. Unlike TCP, the data packets are sent immediately without a prior handshake protocol.

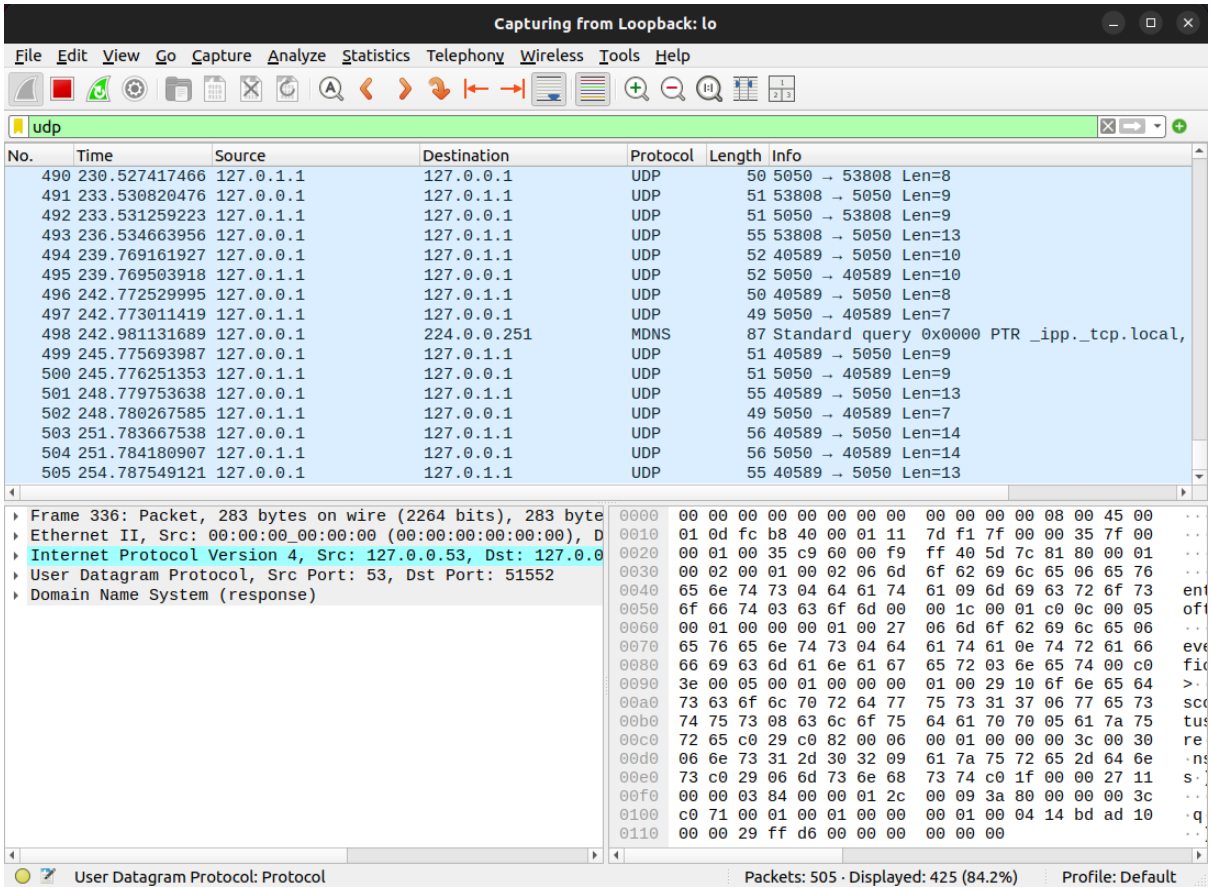

Figure 4: Wireshark capture showing isolated UDP datagrams.

5 Conclusion

Through this lab, the fundamental differences between TCP and UDP were observed practically. TCP provided reliable, ordered data streams requiring explicit connection handling, whereas UDP allowed for faster, connectionless transmission at the application layer.